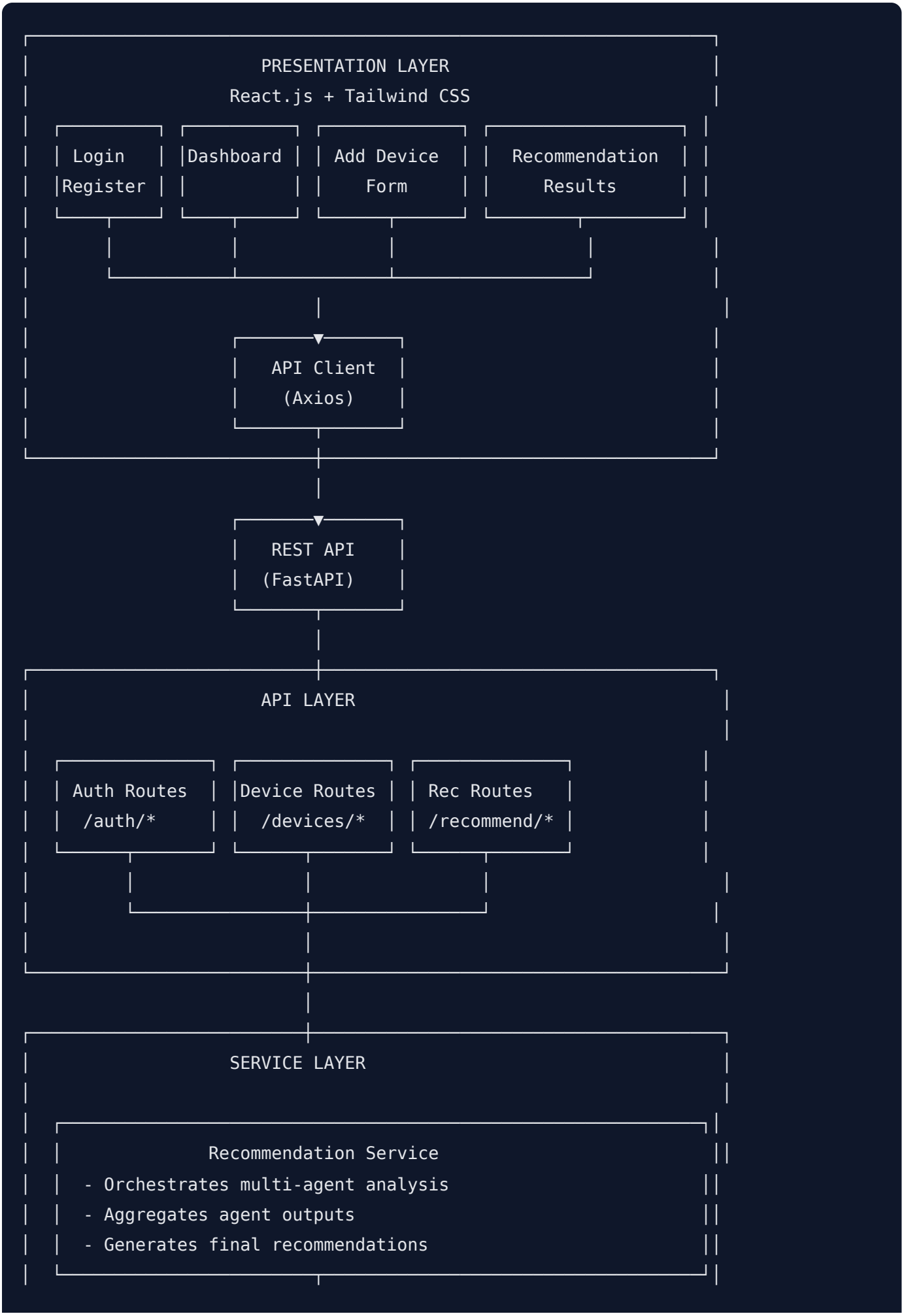
System Architecture

Overview

The Sustainable E-Waste Collection Interface uses a modern microservices-based architecture with a multi-agent AI system for personalized disposal recommendations.


PRESENTATION LAYER
React.js + Tailwind CSS
Login
Register
Dashboard
Add Device
Form
Recommendation
Results
API Client
(Axios)
REST API
(FastAPI)
API LAYER
Auth Routes
/auth/*
Device Routes
/devices/*
Rec Routes
/recommend/*
SERVICE LAYER
Recommendation Service
- Orchestrates multi-agent analysis
- Aggregates agent outputs
- Generates final recommendations


MULTI-AGENT SYSTEM
EVALUATION AGENT
• Resale Value
• Market Analysis
SUSTAINABILITY AGENT
• Eco Score
• CO2 Impact
• Hazard Assessment
LOCATION AGENT
• Find Centers
• Distance Calc
DECISION AGENT
• Rule Engine
• Final Decision
• Confidence
Location Service
- Center management
- Geo-distance calculations
- Google Maps integration
DATA LAYER
MongoDB Database
Users
Devices
Recommendations
Centers

Component Descriptions

Frontend (React.js)

Component	Purpose
`App.js`	Main router, authentication context
`Navbar.jsx`	Navigation, user menu
`DeviceForm.jsx`	Multi-step device input form
`RecommendationCard.jsx`	Displays AI recommendation results
`MapComponent.jsx`	Google Maps integration

Backend (FastAPI)

Module	Purpose
`main.py`	Application entry point, CORS, routes
`config.py`	Environment configuration
`database.py`	MongoDB connection, indexes
`routes/`	API endpoint definitions
`services/`	Business logic layer
`agents/`	Multi-agent AI system
`utils/`	Helper functions

Multi-Agent System


INPUT: Device Data
(type, brand, model, age, condition, working status)
EVALUATION AGENT
Output:
- value: $
- factors[]
SUSTAINABILITY AGENT
Output:
- score: 0-1
- co2: kg
LOCATION AGENT
Output:
- centers[]
- distances
DECISION AGENT
Rules Engine:
1. Check value
2. Check eco
3. Check centers
4. Make decision
OUTPUT: Recommendation
{
recommendation: "recycle|resell|refurbish|donate",
confidence: 0.85,
evaluation_score: 150.00,
sustainability_score: 0.72,
co2_impact: 45.5,
nearby_centers: [...],
reasoning: "..."

```
|  }                                                                  |
└─────────────────────────────────────────────────────────────────────┘
```

Data Flow

1. User Registration/Login

```
User → Frontend → POST /auth/register|login → JWT Token → localStorage
```

2. Add Device

```
User → DeviceForm → POST /devices → MongoDB → Device ID
```

3. Get Recommendation

```
User → Click "Recommend" → GET /recommend/{device_id}
  → Evaluation Agent (calculates value)
  → Sustainability Agent (calculates eco score)
  → Location Agent (finds centers)
  → Decision Agent (makes final decision)
  → Store in MongoDB
  → Return to Frontend
```

4. Find Centers

```
User → MapView → GET /centers/nearby?lat=X&lng=Y
  → MongoDB geo query
  → Return sorted by distance
  → Display on Google Maps
```

Database Schema

Users Collection

```
{
  _id: ObjectId,
  email: String (unique),
  name: String,
  hashed_password: String,
  location: {
    lat: Number,
    lng: Number,
    address: String
  },
  created_at: Date,
  updated_at: Date
}
```

Devices Collection

```
{
  _id: ObjectId,
  user_id: ObjectId (ref: users),
  device_type: String (enum),
  brand: String,
  model_name: String,
  age: Number,
  condition: String (enum),
  is_working: Boolean,
  has_original_accessories: Boolean,
  description: String,
  purchase_year: Number,
  has_recommendation: Boolean,
  created_at: Date,
  updated_at: Date
}
```

Recommendations Collection

```
{
  _id: ObjectId,
  device_id: ObjectId (ref: devices),
  user_id: ObjectId (ref: users),
  recommendation: String (enum: recycle|resell|refurbish|donate),
  confidence: Number (0-1),
  evaluation_score: Number,
  sustainability_score: Number,
  co2_impact: Number,
  reasoning: String,
  agent_outputs: {
    evaluation: Object,
    sustainability: Object,
    location: Object,
    decision: Object
  },
  created_at: Date
}
```

Centers Collection

```
{
  _id: ObjectId,
  name: String,
  address: String,
  lat: Number,
  lng: Number,
  phone: String,
  website: String,
  operating_hours: String,
  accepted_device_types: [String],
  is_certified: Boolean,
  certification_details: String,
  created_at: Date
}
```

API Endpoints

Authentication

Method	Endpoint	Description
POST	`/auth/register`	Register new user
POST	`/auth/login`	Login user
GET	`/auth/me`	Get current user

Devices

Method	Endpoint	Description
GET	`/devices`	List user's devices
POST	`/devices`	Create device
GET	`/devices/{id}`	Get device
DELETE	`/devices/{id}`	Delete device
GET	`/devices/stats`	Get device statistics

Recommendations

Method	Endpoint	Description
GET	`/recommend/{device_id}`	Get/generate recommendation
GET	`/recommendations`	List user's recommendations

Centers

Method	Endpoint	Description
GET	`/centers`	List all centers
GET	`/centers/nearby`	Find nearby centers
POST	`/centers`	Add center

Security

- **JWT Authentication**: Token-based auth with 7-day expiry
- **Password Hashing**: bcrypt with salt
- **CORS**: Configured for specific origins
- **Input Validation**: Pydantic models for all inputs

Scalability Considerations

1. **Database Indexing**: Geo-spatial indexes for center queries
2. **Async Operations**: Motor driver for non-blocking DB ops
3. **Stateless API**: Easy horizontal scaling
4. **Agent Modularity**: Agents can be distributed/scaled independently